

BUILD-SUPABASE.md

Partie C.4 — Types de données PostgreSQL

Version : V1

Statut : Référence officielle des types de données PostgreSQL

1. Objectif

Cette section définit les types de données autorisés dans PostgreSQL pour Vitala.

Elle garantit :

- la cohérence du schéma de données ;
- la compatibilité entre les domaines ;
- des performances optimales ;
- une maintenance simplifiée.

Le choix d'un type de données doit toujours être justifié par le besoin métier.

2. Principes directeurs

Le choix des types de données repose sur les principes suivants :

- précision ;
- cohérence ;
- évolutivité ;
- lisibilité ;
- performance.

Le type le plus adapté doit toujours être privilégié.

3. UUID

Le type `UUID` est utilisé pour tous les identifiants métier.

Utilisations :

- clés primaires ;
- clés étrangères ;
- identifiants publics.

Avantages :

- unicité globale ;
- sécurité accrue ;
- compatibilité avec la synchronisation Offline.

Les identifiants auto-incrémentés sont réservés aux besoins techniques exceptionnels.

4. TEXT

Le type `TEXT` est le type par défaut pour les contenus textuels dont la longueur n'est pas limitée.

Exemples :

- descriptions ;
- commentaires ;
- biographies ;
- contenus rédigés.

`TEXT` est préféré à `VARCHAR` lorsqu'aucune limite métier n'est nécessaire.

5. VARCHAR

`VARCHAR(n)` est utilisé uniquement lorsqu'une longueur maximale fait partie de la règle métier.

Exemples :

- code pays ;
- code de langue ;
- numéro de version ;
- identifiant court.

La longueur maximale doit être documentée.

6. BOOLEAN

Le type `BOOLEAN` représente uniquement deux états :

- vrai ;
- faux.

Il ne doit pas être utilisé pour représenter plusieurs états métier.

7. INTEGER

Le type `INTEGER` est utilisé pour les valeurs numériques entières de taille standard.

Exemples :

- compteurs ;
 - quantités ;
 - durées en secondes.
-

8. BIGINT

`BIGINT` est réservé aux valeurs pouvant dépasser les limites d'un `INTEGER`.

Exemples :

- statistiques volumineuses ;
 - compteurs globaux.
-

9. NUMERIC

Le type `NUMERIC` est utilisé lorsqu'une précision exacte est indispensable.

Exemples :

- montants financiers ;
 - taux ;
 - pourcentages ;
 - scores nécessitant une précision décimale.
-

10. DATE

Le type `DATE` est utilisé lorsqu'une heure n'est pas nécessaire.

Exemples :

- date de naissance ;
 - échéance ;
 - anniversaire.
-

11. TIMESTAMPTZ

Le type `TIMESTAMPTZ` est le standard officiel pour tous les événements horodatés.

Exemples :

- `created_at`
- `updated_at`
- `deleted_at`
- `last_login_at`

Toutes les dates et heures doivent être stockées avec le fuseau horaire.

12. JSONB

Le type `JSONB` est réservé aux données semi-structurées.

Exemples :

- préférences utilisateur ;
- métadonnées ;
- paramètres configurables ;
- informations provenant d'intégrations externes.

Il ne doit pas remplacer une modélisation relationnelle lorsque celle-ci est plus adaptée.

13. ARRAY

Les tableaux (`ARRAY`) sont autorisés uniquement lorsque :

- le nombre d'éléments reste limité ;
- les éléments ne nécessitent pas de relations complexes.

Sinon, une table dédiée est préférable.

14. ENUM

Les `ENUM` sont utilisés pour des listes fermées de valeurs.

Exemples :

- statuts ;
- niveaux ;
- catégories fixes.

Toute évolution d'un `ENUM` doit être accompagnée d'une migration.

15. Types interdits ou à éviter

Les pratiques suivantes sont déconseillées :

- utilisation systématique de `VARCHAR(255)` sans justification ;
 - stockage de données relationnelles dans un champ `JSONB` ;
 - utilisation de `TEXT` pour représenter des nombres ;
 - multiplication de colonnes génériques (`value`, `data`, `info`) sans signification métier.
-

16. Évolution des types

Le changement d'un type de données doit :

- être précédé d'une analyse d'impact ;
 - être réalisé via une migration ;
 - préserver les données existantes ;
 - être documenté.
-

17. Documentation

Chaque colonne doit préciser :

- son type ;
 - sa signification métier ;
 - ses contraintes éventuelles ;
 - sa valeur par défaut si applicable.
-

18. Critères de conformité

L'utilisation des types est conforme lorsque :

- le type choisi est adapté au besoin métier ;
 - les conventions sont respectées ;
 - les migrations sont documentées ;
 - les performances restent satisfaisantes ;
 - la documentation est synchronisée.
-

19. Conclusion

Le choix des types de données constitue un élément fondamental de la qualité de la base PostgreSQL de Vitala.

Le respect de ces conventions garantit une base de données cohérente, performante, évolutive et compatible avec les principes du Build Blueprint.